

Figure reference: working code, lessons learned, and notes

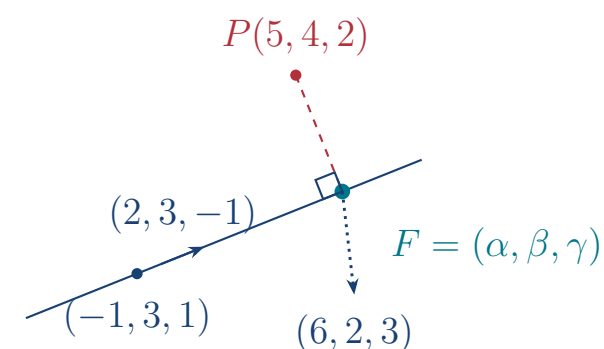
This page is a working reference for building the remaining figures across this file, not a problem to solve. It exists because getting one figure right (below) took many rounds of iteration, and that pace doesn't scale to every problem — so the working code and the lessons that came out of it are banked here, to be reused directly rather than re-learned from scratch each time.

Lessons learned, in the order they bit

1. **Verify every “on-line” point numerically before drawing it.** A schematic line's endpoints were hand-picked without checking they actually passed through the labelled points — it looked fine, it wasn't. Fix: pick a base point and direction, then compute every other point on that line as $A + t\vec{d}$ for some t , and verify algebraically (t from the x -match must equal t from the y -match) before trusting the drawing. This applies to every point claimed to lie on a line: endpoints, ticks, everything.
2. **Label collisions need explicit anchors, not offset-shifts.** `xshift/yshift` alone just nudge a *center*-anchored box, so half the text can still sit back over the point it's labelling. Use `anchor=south`, `anchor=north`, `west`, etc. to control which *edge* of the text sits at the reference point — that's what actually stops the overlap.
3. **Font size: don't wrap figure labels in `\small` or similar.** Labels should inherit the ambient document font directly, matching body text. If the drawing then looks too large relative to properly-sized text, shrink the `scale` parameter — don't shrink the text.
4. **Two elements meeting at a point need genuinely different visual cues, not just different angles.** An arrow at a shallow angle from another line can still read as “possibly the same line continuing,” especially describing 3D on a 2D page. Combine color *and* line style (solid / dashed / dotted) so the distinction holds regardless of the exact angle chosen.
5. **Color by role, not by arbitrary choice.** In the worked example below: red = the given external point, blue/navy = all given fixed data (a line's point, its direction, a target direction to project onto), and a third color = the computed/unknown quantity being solved for. Reusing this convention across figures means a student learns the color code once, not per-figure.
6. **A `tikzpicture` in a `[t]`-aligned minipage needs `baseline=(current bounding box.north)`.** Without it, TikZ aligns by the *bottom* of its own bounding box by default, so a figure can visibly start higher or lower than adjacent text even though both minipages are top-aligned.
7. **That baseline fix alone is usually not enough.** `\centering` still tends to leave a fixed gap (around 40pt was measured for one figure, at this file's font size) before the picture starts. Measure the actual pixel gap between the text column's first ink and the figure's first ink at high render resolution, convert to points, and cancel it with `\vspace{-Xpt}` right before `\begin{tikzpicture}`. This value is specific to each figure's content and this document's font size — it needs re-measuring per figure, not copying from one to the next.

Worked example: M26S1J21Q64, fully resolved

Given: (α, β, γ) is the foot of the perpendicular from $(5, 4, 2)$ to the line $\vec{r} = (-\hat{i} + 3\hat{j} + \hat{k}) + \lambda(2\hat{i} + 3\hat{j} - \hat{k})$. **Find:** the projection of $\alpha\hat{i} + \beta\hat{j} + \gamma\hat{k}$ onto $6\hat{i} + 2\hat{j} + 3\hat{k}$.



Notes for future figures